informe final desafio II

UdeABook.

Oscar Mauricio Echeverri Parra.

Luna Jaramillo Guzmán.

En este proyecto desarrollamos Ude@Book, una aplicación en C++ que simula algunas de las funciones principales de una red social. Permite gestionar usuarios, enviar solicitudes de amistad y mensajes, y compartir fotografías mediante álbumes.
Uno de los mayores retos fue lograr que todas las clases y funciones trabajaran en conjunto. Más que implementar cada función por separado, tuvimos que asegurarnos de que la información estuviera bien conectada y de que las acciones de cada usuario se reflejaran correctamente en el resto de la aplicación. A lo largo del proceso, fuimos resolviendo estos desafíos y aprendiendo la importancia de diseñar un programa cuyos componentes puedan comunicarse entre sí.

Diseño de las clases y organización del programa.

Organizamos Ude@Book en distintas clases, cada una encargada de representar una parte importante de la aplicación y cumplir una función específica. Como muestra el diagrama de clases, estas se relacionan y se comunican entre sí para que todo el sistema funcione de manera unificada.

La clase Plataforma es el núcleo de la aplicación, pues coordina su funcionamiento general. Desde ella se gestionan los usuarios, los mensajes, las solicitudes de amistad, los archivos y el uso de recursos. Así, las principales operaciones de Ude@Book se organizan de manera centralizada.

Clases principales.

Plataforma

Se encarga de coordinar el funcionamiento general del sistema. Gestiona las listas de usuarios, mensajes, solicitudes y mantiene el control de la sesión activa. Además, incluye funciones para cargar y guardar información, buscar usuarios, mostrar las fotografías de los amigos y presentar el menú principal.

Usuario

Representa a cada persona registrada en la red social y guarda tanto sus datos personales como la información vinculada a su actividad. Por ejemplo, mantiene sus listas de amigos, álbumes, mensajes enviados y recibidos, y solicitudes de amistad. También reúne las funciones para gestionar solicitudes, encontrar amigos en común, sugerir nuevos contactos, enviar mensajes y consultar álbumes.

SolicitudAmistad

Representa una petición de amistad entre un usuario que la envía y otro que la recibe. También registra su estado y permite aceptar o rechazar la solicitud.

Mensaje

Guarda la información de cada mensaje: quién lo envía, quién lo recibe, cuándo se envió, su contenido y si ya fue leído. Además, permite actualizar su estado a leído y verificar si se envió dentro de un intervalo de fechas determinado.

Album y Foto

Reúne las fotografías de un usuario y guarda datos como el nombre, la ruta, la configuración de privacidad y la lista de fotos. Por su parte, la clase Foto conserva detalles de cada imagen como la fecha de captura, el tamaño y la ruta y permite agregar o quitar «me gusta».

clases auxilares.

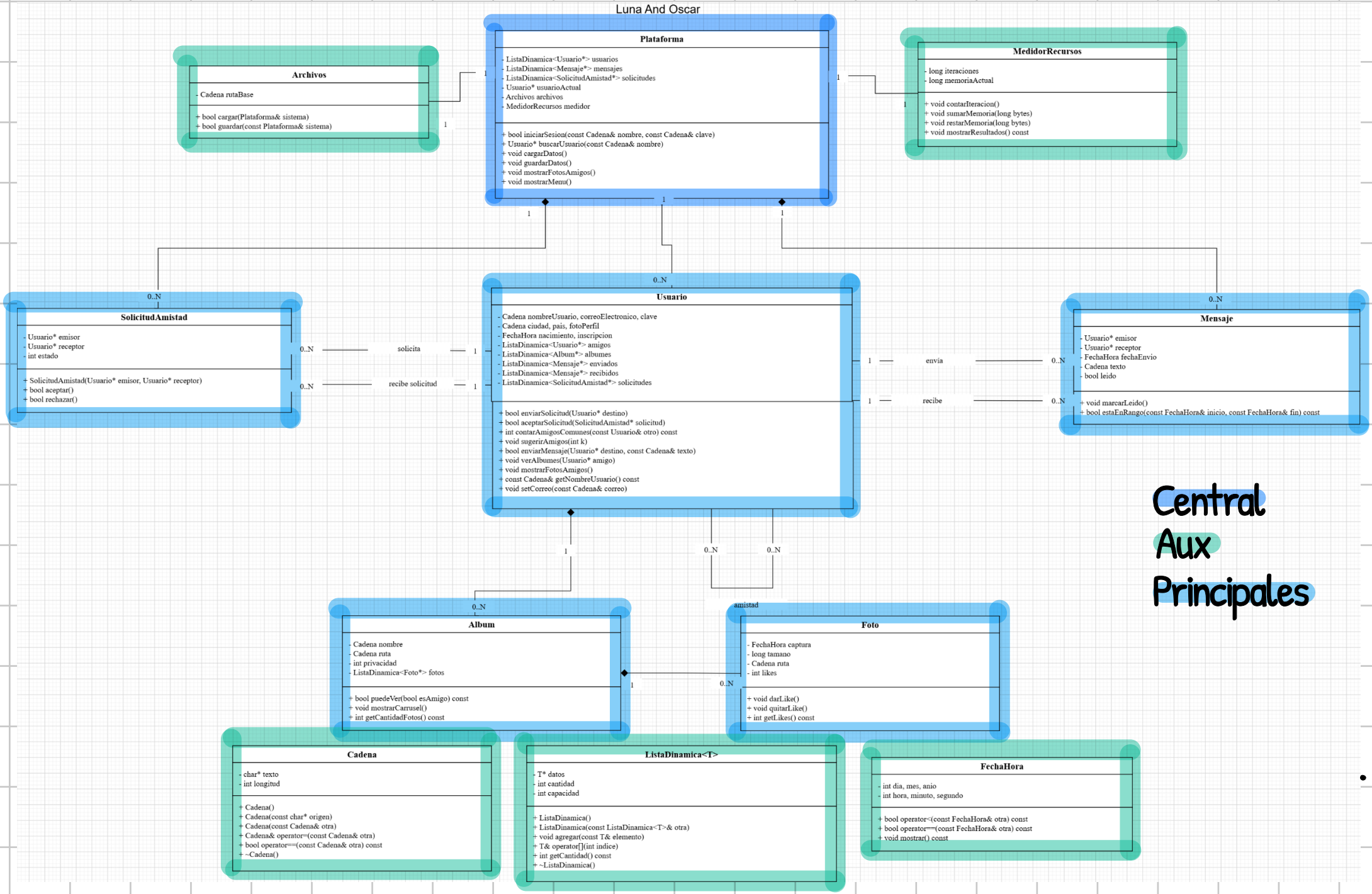
El diagrama también muestra varias clases que ayudan a que la aplicación funcione correctamente:
ListaDinamica<T> permite almacenar colecciones de elementos en una estructura que puede crecer o reducirse según sea necesario.
Cadena reúne las funciones para trabajar con texto.
FechaHora representa fechas y horas, y permite compararlas; por ejemplo, para buscar mensajes enviados dentro de un período determinado.
Archivos se encarga de leer y guardar la información que debe conservarse.
MedidorRecursos registra las iteraciones y el uso de memoria reportados por el programa.

Relación de las clases.

Las relaciones del diagrama muestran cómo se conectan las distintas partes de Ude@Book. La Plataforma organiza las colecciones generales, mientras que cada Usuario gestiona sus amistades, solicitudes, mensajes y álbumes. A su vez, cada álbum reúne las fotografías que lo componen.

Las solicitudes de amistad y los mensajes vinculan a usuarios ya registrados. Así, el sistema puede identificar quién envía o recibe una solicitud y quiénes participan en una conversación, sin crear representaciones adicionales de esas personas.

En conjunto, el diagrama ayuda a entender cómo se reparten las responsabilidades y cómo colaboran las clases para ofrecer las funciones de la aplicación.


Luna And Oscar
Archivos
- Cadena rutaBase
+ bool cargar(Plataforma& sistema)
+ bool guardar(const Plataforma& sistema)
Plataforma
- ListaDinamica<Usuario*> usuarios
- ListaDinamica<Mensaje*> mensajes
- ListaDinamica<SolicitudAmistad*> solicitudes
- Usuario* usuarioActual
- Archivos archivos
- MedidorRecursos medidor
+ bool iniciarSesion(const Cadena& nombre, const Cadena& clave)
+ Usuario* buscarUsuario(const Cadena& nombre)
+ void cargarDatos()
+ void guardarDatos()
+ void mostrarFotosAmigos()
+ void mostrarMenu()
MedidorRecursos
- long iteraciones
- long memoriaActual
+ void contarIteracion()
+ void sumarMemoria(long bytes)
+ void restarMemoria(long bytes)
+ void mostrarResultados() const
SolicitudAmistad
- Usuario* emisor
- Usuario* receptor
- int estado
+ SolicitudAmistad(Usuario* emisor, Usuario* receptor)
+ bool aceptar()
+ bool rechazar()
solicita
recibe solicitud
Usuario
- Cadena nombreUsuario, correoElectronico, clave
- Cadena ciudad, pais, fotoPerfil
- FechaHora nacimiento, inscripcion
- ListaDinamica<Usuario*> amigos
- ListaDinamica<Album*> albumes
- ListaDinamica<Mensaje*> enviados
- ListaDinamica<Mensaje*> recibidos
- ListaDinamica<SolicitudAmistad*> solicitudes
+ bool enviarSolicitud(Usuario* destino)
+ bool aceptarSolicitud(SolicitudAmistad* solicitud)
+ int contarAmigosComunes(const Usuario& otro) const
+ void sugerirAmigos(int k)
+ bool enviarMensaje(Usuario* destino, const Cadena& texto)
+ void verAlbumes(Usuario* amigo)
+ void mostrarFotosAmigos()
+ const Cadena& getNombreUsuario() const
+ void setCorreo(const Cadena& correo)
envía
recibe
Mensaje
- Usuario* emisor
- Usuario* receptor
- FechaHora fechaEnvio
- Cadena texto
- bool leido
+ void marcarLeido()
+ bool estaEnRango(const FechaHora& inicio, const FechaHora& fin) const
Album
- Cadena nombre
- Cadena ruta
- int privacidad
- ListaDinamica<Foto*> fotos
+ bool puedeVer(bool esAmigo) const
+ void mostrarCarrusel()
+ int getCantidadFotos() const
amistad
Foto
- FechaHora captura
- long tamano
- Cadena ruta
- int likes
+ void darLike()
+ void quitarLike()
+ int getLikes() const
Cadena
- char* texto
- int longitud
+ Cadena()
+ Cadena(const char* origen)
+ Cadena(const Cadena& otra)
+ Cadena& operator=(const Cadena& otra)
+ bool operator==(const Cadena& otra) const
+ ~Cadena()
ListaDinamica<T>
- T* datos
- int cantidad
- int capacidad
+ ListaDinamica()
+ ListaDinamica(const ListaDinamica<T>& otra)
+ void agregar(const T& elemento)
+ T& operator[](int indice)
+ int getCantidad() const
+ ~ListaDinamica()
FechaHora
- int dia, mes, anio
- int hora, minuto, segundo
+ bool operator<(const FechaHora& otra) const
+ bool operator==(const FechaHora& otra) const
+ void mostrar() const
Central
Aux
Principales

desarrollo e implementación.

Organización e implementación de las clases

El programa está organizado en archivos de encabezado (.h) y de implementación (.cpp). Esta separación permite declarar las clases y sus métodos en un lugar, y desarrollar su funcionamiento en otro, lo que facilita el orden y mantenimiento del código.
La clase Plataforma se encarga de coordinar las funciones generales de la aplicación, como cargar y guardar datos, iniciar sesión, buscar usuarios, gestionar amistades, enviar mensajes y mostrar sugerencias.
Por su parte, Usuario reúne las acciones propias de cada persona, relacionadas con sus amistades, álbumes, mensajes y solicitudes.
SolicitudAmistad y Mensaje representan las interacciones entre usuarios, mientras que Album y Foto permiten organizar y consultar las imágenes.

Organización y almacenamiento de la información

La plantilla ListaDinamica se utiliza para gestionar colecciones: permite agregar, insertar y eliminar elementos, y ajustar el espacio de almacenamiento según se necesite.
La clase Archivos se ocupa de cargar y guardar los datos de la plataforma.
Cadena y FechaHora, en cambio, ofrecen herramientas para trabajar con texto y fechas, necesarias para funciones como enviar mensajes o buscar aquellos enviados durante un período específico.

Implementación de las funcionalidades

Las funciones de Ude@Book se organizan en torno a las opciones del menú:
Inicio de sesión y perfiles: permite identificar al usuario y ver las fotografías de sus amigos.

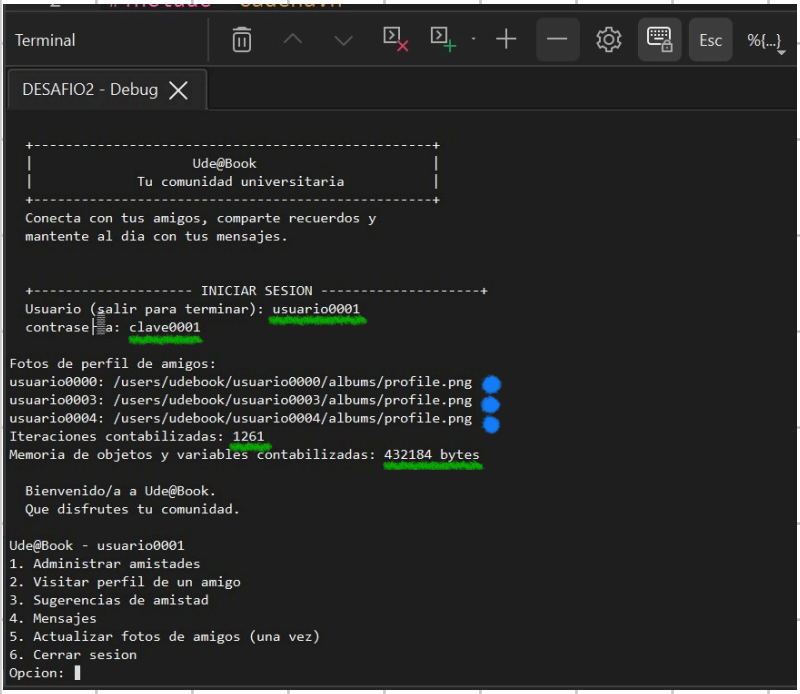
Amistades y sugerencias: permite enviar y gestionar solicitudes, además de encontrar posibles amigos a partir de los contactos en común.
Álbumes y fotografías: permite consultar álbumes a los que se tiene acceso, recorrer sus fotos y registrar «me gusta».
Mensajes: permite enviar mensajes, revisar los que aún no se han leído y buscar conversaciones por fecha.
Los métodos de las distintas clases conectan estas funciones y permiten que cada una trabaje con la información que administran los componentes correspondientes.

Generación de datos de prueba

Para probar Ude@Book, usamos datos de texto generados con inteligencia artificial. Así contamos con suficientes usuarios, mensajes y otros elementos para poner a prueba las funciones de la aplicación.
Con estos datos simulamos distintas interacciones y verificamos que procesos como enviar y recibir mensajes, gestionar solicitudes de amistad y buscar mensajes por fecha funcionaran correctamente.

Pruebas de funcionamiento.

Inicio de sesión y visualización de fotografías de amigos.



Envío y aceptación de solicitudes de amistad.

```
Ude@Book - usuario0001
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 1
1. Enviar solicitud
2. Ver solicitudes recibidas
Opcion: 1
Nombre del usuario: usuario0099
Solicitud enviada.
Iteraciones contabilizadas: 100
Memoria de objetos y variables contabilizadas: 432240 bytes
```

```
  +-------------------- INICIAR SESION --------------------+
  Usuario (salir para terminar): usuario0099
  contrase▒a: clave0099

Fotos de perfil de amigos:
usuario0102: /users/udebook/usuario0102/albums/profile.png
usuario0098: /users/udebook/usuario0098/albums/profile.png
usuario0096: /users/udebook/usuario0096/albums/profile.png
Iteraciones contabilizadas: 1193
Memoria de objetos y variables contabilizadas: 432240 bytes

  Bienvenido/a a Ude@Book.
  Que disfrutes tu comunidad.

Ude@Book - usuario0099
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 1
1. Enviar solicitud
2. Ver solicitudes recibidas
Opcion: 2
1. usuario0001
Numero (0 volver): ▌
```

```
  Bienvenido/a a Ude@Book.
  Que disfrutes tu comunidad.

Ude@Book - usuario0099
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 1
1. Enviar solicitud
2. Ver solicitudes recibidas
Opcion: 2
1. usuario0001
Numero (0 volver): 1
1. Aceptar  2. Rechazar: 1
Solicitud procesada.
Iteraciones contabilizadas: 1
Memoria de objetos y variables contabilizadas: 432240 bytes
```

Consulta de las 12 sugerencias.

```
  Bienvenido/a a Ude@Book.
  Que disfrutes tu comunidad.

Ude@Book - usuario0001
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 3
Cantidad K (1 a 20): 12
1. usuario0005 (4 amigos en comun)
2. usuario0006 (3 amigos en comun)
3. usuario0102 (3 amigos en comun)
4. usuario0997 (3 amigos en comun)
5. usuario0007 (2 amigos en comun)
6. usuario0098 (2 amigos en comun)
7. usuario0100 (2 amigos en comun)
8. usuario0103 (2 amigos en comun)
9. usuario0104 (2 amigos en comun)
10. usuario0996 (2 amigos en comun)
11. usuario0008 (1 amigos en comun)
12. usuario0009 (1 amigos en comun)
Enviar solicitud a cual numero (0 para volver):
```

```
Enviar solicitud a cual numero (0 para volver): 1
Solicitud enviada.
Iteraciones contabilizadas: 3104
Memoria de objetos y variables contabilizadas: 432328 bytes
```

→ Cambio.

Búsqueda de mensajes: 0 resultados con usuario0001 y 84 con usuario0000.

```
Ude@Book - usuario0001
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 4
1. Enviar mensaje
2. Leer nuevos
3. Buscar por fechas
Opcion: 3
Fecha inicial AAAA-MM-DD: 2026-07-01
Fecha final AAAA-MM-DD: 2026-08-12
Mensajes encontrados: 0
Iteraciones contabilizadas: 0
Memoria de objetos y variables contabilizadas: 432328 bytes

Ude@Book - usuario0001
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion:
```

```
  +-------------------- INICIAR SESION --------------------+
  Usuario (salir para terminar): usuario0000
  contrase▒a: clave0000

Fotos de perfil de amigos:
usuario0003: /users/udebook/usuario0003/albums/profile.png
usuario0004: /users/udebook/usuario0004/albums/profile.png
usuario0007: /users/udebook/usuario0007/albums/profile.png
Iteraciones contabilizadas: 17545
Memoria de objetos y variables contabilizadas: 432328 bytes

  Bienvenido/a a Ude@Book.
  Que disfrutes tu comunidad.

Ude@Book - usuario0000
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 4
1. Enviar mensaje
2. Leer nuevos
3. Buscar por fechas
Opcion: 3
Fecha inicial AAAA-MM-DD: 2026-07-01
Fecha final AAAA-MM-DD: 2026-08-12
2026-07-01T00:00:00 | usuario0001: Mensaje de prueba numero 1
2026-07-01T00:00:00 | usuario0013: Mensaje de prueba numero 13
2026-07-01T00:00:00 | usuario0025: Mensaje de prueba numero 25
```

```
2026-07-01T00:00:00 | usuario0037: Mensaje de prueba numero 37
2026-07-01T00:00:00 | usuario0049: Mensaje de prueba numero 49
2026-07-01T00:00:00 | usuario0061: Mensaje de prueba numero 61
2026-07-02T00:00:00 | usuario0002: Mensaje de prueba numero 2
2026-07-02T00:00:00 | usuario0014: Mensaje de prueba numero 14
2026-07-02T00:00:00 | usuario0026: Mensaje de prueba numero 26
2026-07-02T00:00:00 | usuario0038: Mensaje de prueba numero 38
2026-07-02T00:00:00 | usuario0050: Mensaje de prueba numero 50
2026-07-02T00:00:00 | usuario0062: Mensaje de prueba numero 62
2026-07-03T00:00:00 | usuario0003: Mensaje de prueba numero 3
2026-07-03T00:00:00 | usuario0015: Mensaje de prueba numero 15
2026-07-03T00:00:00 | usuario0027: Mensaje de prueba numero 27
2026-07-03T00:00:00 | usuario0039: Mensaje de prueba numero 39
2026-07-03T00:00:00 | usuario0051: Mensaje de prueba numero 51
2026-07-03T00:00:00 | usuario0063: Mensaje de prueba numero 63
2026-07-04T00:00:00 | usuario0004: Mensaje de prueba numero 4
2026-07-04T00:00:00 | usuario0016: Mensaje de prueba numero 16
2026-07-04T00:00:00 | usuario0028: Mensaje de prueba numero 28
2026-07-04T00:00:00 | usuario0040: Mensaje de prueba numero 40
2026-07-04T00:00:00 | usuario0052: Mensaje de prueba numero 52
2026-07-04T00:00:00 | usuario0064: Mensaje de prueba numero 64
2026-07-05T00:00:00 | usuario0005: Mensaje de prueba numero 5
2026-07-05T00:00:00 | usuario0017: Mensaje de prueba numero 17
2026-07-05T00:00:00 | usuario0029: Mensaje de prueba numero 29
2026-07-05T00:00:00 | usuario0041: Mensaje de prueba numero 41
2026-07-05T00:00:00 | usuario0053: Mensaje de prueba numero 53
2026-07-05T00:00:00 | usuario0065: Mensaje de prueba numero 65
2026-07-06T00:00:00 | usuario0006: Mensaje de prueba numero 6
2026-07-06T00:00:00 | usuario0018: Mensaje de prueba numero 18
2026-07-06T00:00:00 | usuario0030: Mensaje de prueba numero 30
2026-07-06T00:00:00 | usuario0042: Mensaje de prueba numero 42
```

```
2026-07-06T00:00:00 | usuario0054: Mensaje de prueba numero 54
2026-07-06T00:00:00 | usuario0066: Mensaje de prueba numero 66
2026-07-07T00:00:00 | usuario0007: Mensaje de prueba numero 7
2026-07-07T00:00:00 | usuario0019: Mensaje de prueba numero 19
2026-07-07T00:00:00 | usuario0031: Mensaje de prueba numero 31
2026-07-07T00:00:00 | usuario0043: Mensaje de prueba numero 43
2026-07-07T00:00:00 | usuario0055: Mensaje de prueba numero 55
2026-07-07T00:00:00 | usuario0067: Mensaje de prueba numero 67
2026-07-08T00:00:00 | usuario0008: Mensaje de prueba numero 8
2026-07-08T00:00:00 | usuario0020: Mensaje de prueba numero 20
2026-07-08T00:00:00 | usuario0032: Mensaje de prueba numero 32
2026-07-08T00:00:00 | usuario0044: Mensaje de prueba numero 44
2026-07-08T00:00:00 | usuario0056: Mensaje de prueba numero 56
2026-07-08T00:00:00 | usuario0068: Mensaje de prueba numero 68
2026-07-09T00:00:00 | usuario0009: Mensaje de prueba numero 9
2026-07-09T00:00:00 | usuario0021: Mensaje de prueba numero 21
2026-07-09T00:00:00 | usuario0033: Mensaje de prueba numero 33
2026-07-09T00:00:00 | usuario0045: Mensaje de prueba numero 45
2026-07-09T00:00:00 | usuario0057: Mensaje de prueba numero 57
2026-07-09T00:00:00 | usuario0069: Mensaje de prueba numero 69
2026-07-10T00:00:00 | usuario0010: Mensaje de prueba numero 10
2026-07-10T00:00:00 | usuario0022: Mensaje de prueba numero 22
2026-07-10T00:00:00 | usuario0034: Mensaje de prueba numero 34
2026-07-10T00:00:00 | usuario0046: Mensaje de prueba numero 46
2026-07-10T00:00:00 | usuario0058: Mensaje de prueba numero 58
2026-07-10T00:00:00 | usuario0070: Mensaje de prueba numero 70
2026-07-11T00:00:00 | usuario0011: Mensaje de prueba numero 11
2026-07-11T00:00:00 | usuario0023: Mensaje de prueba numero 23
2026-07-11T00:00:00 | usuario0035: Mensaje de prueba numero 35
2026-07-11T00:00:00 | usuario0047: Mensaje de prueba numero 47
```

```
2026-07-11T00:00:00 | usuario0059: Mensaje de prueba numero 59
2026-07-11T00:00:00 | usuario0071: Mensaje de prueba numero 71
2026-07-12T00:00:00 | usuario0012: Mensaje de prueba numero 12
2026-07-12T00:00:00 | usuario0024: Mensaje de prueba numero 24
2026-07-12T00:00:00 | usuario0036: Mensaje de prueba numero 36
2026-07-12T00:00:00 | usuario0048: Mensaje de prueba numero 48
2026-07-12T00:00:00 | usuario0060: Mensaje de prueba numero 60
2026-07-12T00:00:00 | usuario0072: Mensaje de prueba numero 72
2026-08-01T00:00:00 | usuario0073: Mensaje de prueba numero 73
2026-08-02T00:00:00 | usuario0074: Mensaje de prueba numero 74
2026-08-03T00:00:00 | usuario0075: Mensaje de prueba numero 75
2026-08-04T00:00:00 | usuario0076: Mensaje de prueba numero 76
2026-08-05T00:00:00 | usuario0077: Mensaje de prueba numero 77
2026-08-06T00:00:00 | usuario0078: Mensaje de prueba numero 78
2026-08-07T00:00:00 | usuario0079: Mensaje de prueba numero 79
2026-08-08T00:00:00 | usuario0080: Mensaje de prueba numero 80
2026-08-09T00:00:00 | usuario0081: Mensaje de prueba numero 81
2026-08-10T00:00:00 | usuario0082: Mensaje de prueba numero 82
2026-08-11T00:00:00 | usuario0083: Mensaje de prueba numero 83
2026-08-12T00:00:00 | usuario0084: Mensaje de prueba numero 84
Mensajes encontrados: 84
Iteraciones contabilizadas: 84
Memoria de objetos y variables contabilizadas: 432328 bytes
```

Consulta de álbumes, navegación y «me gusta».

```
Ude@Book - usuario0000
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 2
Nombre de amigo: usuario0001
1. Album1_0 (5 fotos)
2. Album1_1 (5 fotos)
Seleccione album (0 para volver):
```

```
Ude@Book - usuario0000
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 2
Nombre de amigo: usuario0001
1. Album1_0 (5 fotos)
2. Album1_1 (5 fotos)
Seleccione album (0 para volver): 1

Foto 1/5: /users/udebook/usuario0001/albums/Album1_0/2_0.png | Likes: 0
[s]iguiente [p]revia [l]ike [q]uitar like [x]salir: l

Foto 1/5: /users/udebook/usuario0001/albums/Album1_0/2_0.png | Likes: 1
[s]iguiente [p]revia [l]ike [q]uitar like [x]salir: q

Foto 1/5: /users/udebook/usuario0001/albums/Album1_0/2_0.png | Likes: 0
[s]iguiente [p]revia [l]ike [q]uitar like [x]salir: s

Foto 2/5: /users/udebook/usuario0001/albums/Album1_0/2_1.png | Likes: 1
[s]iguiente [p]revia [l]ike [q]uitar like [x]salir: p

Foto 1/5: /users/udebook/usuario0001/albums/Album1_0/2_0.png | Likes: 0
[s]iguiente [p]revia [l]ike [q]uitar like [x]salir: x
Iteraciones contabilizadas: 11
Memoria de objetos y variables contabilizadas: 432328 bytes
```

Actualización de fotografías de perfil.

```
Ude@Book - usuario0000
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 5

Fotos de perfil de amigos:
usuario0011: /users/udebook/usuario0011/albums/profile.png
usuario0013: /users/udebook/usuario0013/albums/profile.png
usuario0010: /users/udebook/usuario0010/albums/profile.png
Iteraciones contabilizadas: 19000
Memoria de objetos y variables contabilizadas: 432328 bytes

Ude@Book - usuario0000
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 5
La actualizacion ya se utilizo en esta sesion.
Iteraciones contabilizadas: 0
Memoria de objetos y variables contabilizadas: 432328 bytes
```

Envío, recepción, lectura y búsqueda de mensajes.

```
Ude@Book - usuario0000
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 4
1. Enviar mensaje
2. Leer nuevos
3. Buscar por fechas
Opcion: 1
Amigo destinatario: usuario0001
Mensaje (maximo 512 caracteres): Hola, estoy comprobando.
Mensaje enviado.
Iteraciones contabilizadas: 2
Memoria de objetos y variables contabilizadas: 432482 bytes
```

→ Cambio

```
Ude@Book - usuario0001
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 4
1. Enviar mensaje
2. Leer nuevos
3. Buscar por fechas
Opcion: 3
Fecha inicial AAAA-MM-DD: 2026-10-15
Fecha final AAAA-MM-DD: 2026-10-15
2026-10-15T19:20:54 | usuario0000: Hola, estoy comprobando.
Mensajes encontrados: 1
Iteraciones contabilizadas: 1
Memoria de objetos y variables contabilizadas: 432482 bytes

Ude@Book - usuario0001
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion:
```

```
Ude@Book - usuario0001
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion: 4
1. Enviar mensaje
2. Leer nuevos
3. Buscar por fechas
Opcion: 2
No hay mensajes nuevos.
Iteraciones contabilizadas: 1
Memoria de objetos y variables contabilizadas: 432482 bytes

Ude@Book - usuario0001
1. Administrar amistades
2. Visitar perfil de un amigo
3. Sugerencias de amistad
4. Mensajes
5. Actualizar fotos de amigos (una vez)
6. Cerrar sesion
Opcion:
```

Inicio de sesión y visualización de fotos de amigos

Para esta prueba, iniciamos sesión con un usuario registrado y verificamos que pudiera acceder al menú principal. Luego, consultamos las fotos de perfil de sus amigos. Así comprobamos que el ingreso funcionara correctamente y que se mostraran las fotografías correspondientes.

Envío y aceptación de solicitudes de amistad

Enviamos una solicitud de amistad desde usuario0001 a usuario0099 y comprobamos que la aplicación confirmara el envío. Después, iniciamos sesión con la cuenta destinataria y aceptamos la solicitud. De esta manera, verificamos que el proceso funcionara correctamente para ambos usuarios.

Consulta de sugerencias de amistad

Consultamos las sugerencias disponibles para usuario0001. La aplicación mostró 12 posibles amistades e indicó cuántos amigos tenían en común. La primera sugerencia compartía cuatro contactos, mientras que las demás tenían entre uno y tres. Con esta prueba verificamos que las sugerencias y la información para compararlas se mostraran correctamente.

Búsqueda de mensajes por intervalo de fechas

Realizamos una búsqueda de mensajes entre el 1 de julio y el 12 de agosto de 2026. Al ejecutar la consulta con usuario0001, no se encontraron mensajes en el intervalo seleccionado. Luego, con usuario0000, se obtuvieron 84 resultados. La comparación permitió comprobar que los resultados de la búsqueda dependían de los mensajes disponibles para cada usuario y de las fechas indicadas.

Consulta de álbumes, navegación y «me gusta»

Desde el perfil de un amigo, consultamos sus álbumes y encontramos album_0 y album_1, cada uno con cinco fotografías. También avanzamos y retrocedimos entre las imágenes, y probamos las opciones para agregar o quitar «me gusta». Así verificamos que la consulta y las interacciones con las fotografías funcionaran correctamente.

Actualización de fotografías de perfil

Ejecutamos la función para actualizar las fotos de perfil y comprobamos que se mostraran tres imágenes de usuarios distintos. En el primer intento, el programa registró 19.000 iteraciones. Al intentar actualizar de nuevo durante la misma sesión, la aplicación indicó que la operación ya se había realizado y reportó cero iteraciones.

Envío, recepción, lectura y búsqueda de mensajes

Enviamos un mensaje y comprobamos que llegara al destinatario. Luego, iniciamos sesión con esa cuenta para consultar el mensaje y verificar su estado de lectura. Por último, confirmamos que pudiera encontrarse mediante la búsqueda correspondiente. Así revisamos el recorrido del mensaje desde su envío hasta su consulta.

Problemas, retos y soluciones

El principal reto de Ude@Book fue integrar sus distintas funciones para que usuarios, amistades, mensajes y fotografías se relacionaran correctamente. Para resolverlo, organizamos el código en clases con responsabilidades claras y probamos las funciones en conjunto con distintos usuarios y datos.

Las pruebas nos permitieron comprobar que las operaciones principales funcionaban como esperábamos y detectar posibles fallos de integración. Así, confirmamos que una buena organización y las pruebas completas fueron clave para superar el desafío.

Conclusión

El desarrollo de Ude@Book nos permitió aplicar conceptos de programación en C++ mediante la creación de una aplicación con funciones propias de una red social. A lo largo del proyecto, trabajamos con usuarios, solicitudes de amistad, mensajes y álbumes de fotografías, organizando cada función en clases con responsabilidades específicas.

Uno de nuestros principales aprendizajes fue comprender cómo relacionar los distintos componentes para que la aplicación funcionara de manera coherente. También realizamos pruebas con diferentes usuarios y situaciones, lo que nos ayudó a verificar las funciones y confirmar que trabajaran correctamente en conjunto.

En general, el proyecto fortaleció nuestros conocimientos de programación orientada a objetos, manejo de estructuras dinámicas y organización de datos. Además, nos mostró que desarrollar una aplicación implica no solo crear funciones individuales, sino también integrarlas para ofrecer una experiencia clara y funcional.